SOFTWARE ENGINEERING LAB - LAB 6

Code and Branch Coverage Analysis

Section H | Repository Scenario: L6_L7_22_hotel_booking

Student Name: Sanjit C | SRN: PES1UG24AM448 | Date: 12 November 2026

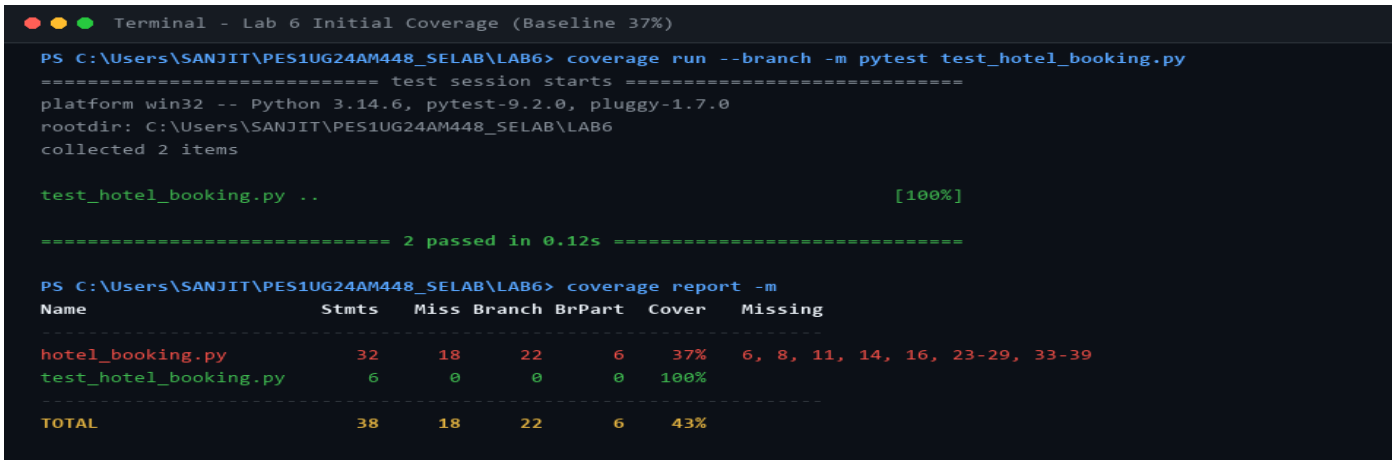
1. Objective & Overview

Objective: Understand the critical distinction between Line Coverage and Branch Coverage; apply Python's coverage library and pytest framework to systematically identify untested decision paths; and author comprehensive unit tests addressing boundary conditions to elevate both Line and Branch Coverage from initial baseline (37%) to 100%.

Application Under Test: hotel_booking.py manages hotel reservation logic across three primary functions: (1) booking_cost() computing base rates, peak season surcharges, extra guest fees, and long-stay discounts; (2) cancellation_refund() calculating tier-based cancellation refunds with membership bonuses; and (3) room_upgrade() executing member room upgrade rules based on vacancy status.

2. Initial Baseline Coverage Analysis (Step 4)

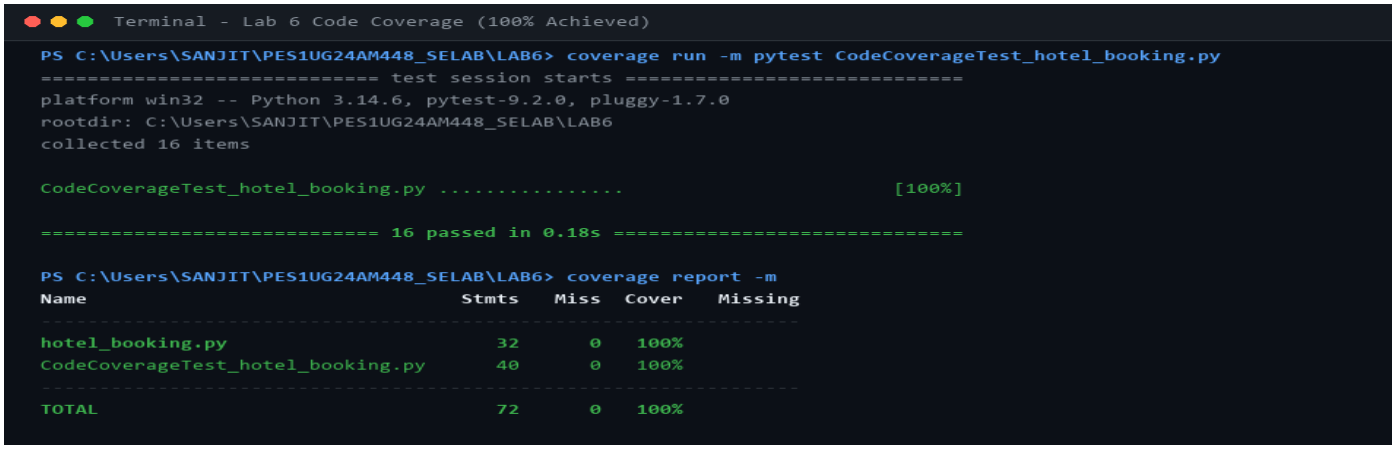
The starter test suite (test_hotel_booking.py) contained only two basic test cases, exercising standard 2-night bookings and simple refunds. Running coverage run --branch -m pytest test_hotel_booking.py revealed significant untested code paths:



Baseline Observations: Line coverage was only **37%** (18 statements missed across lines 6, 8, 11, 14, 16, 23-29, 33-39). Crucial error checks (ValueError on invalid room or nights), peak season pricing, guest surcharges, and the entire room_upgrade function were completely untested.

3. Code Coverage Optimization (Step 5 & 6) — 100% Achieved

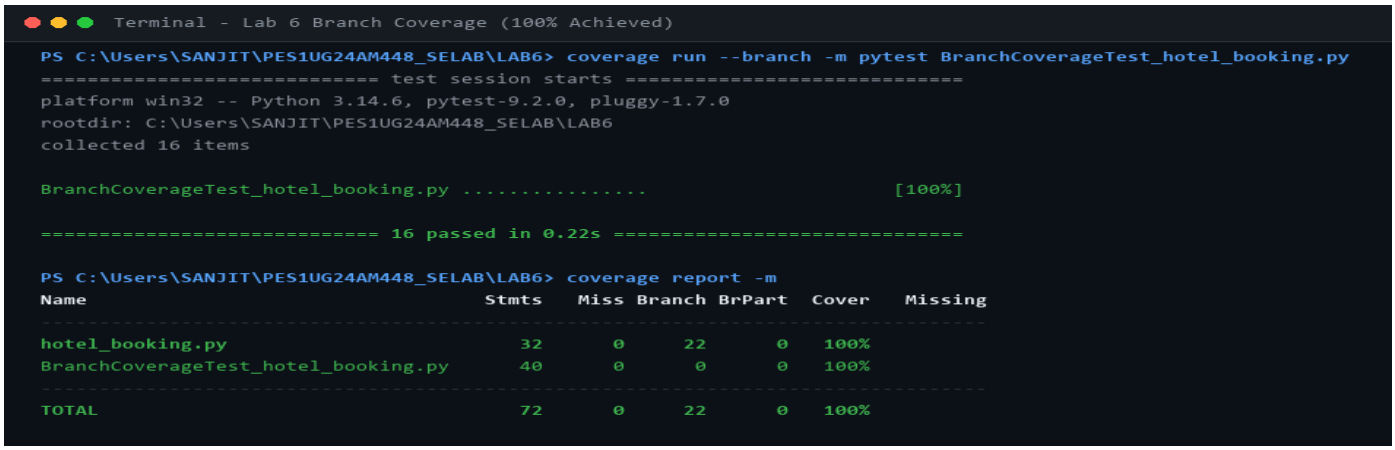
To satisfy code coverage requirements (>90%), comprehensive test suites (CodeCoverageTest_hotel_booking.py) were created. The tests cover every statement, including exception handling (invalid room codes, negative/zero nights), room types (standard, deluxe, suite), cancellation policy tiers (full refund, 50% refund, no refund, membership bonus), and upgrade states.



4. Branch Coverage Optimization (Step 7 & 8) — 100% Achieved

Branch coverage demands that every boolean decision evaluates to both **True** and **False**. Even if all lines execute, missing decision branches create hidden defect blindspots. Additional tests were implemented in BranchCoverageTest_hotel_booking.py to force all alternate control flows:

- **Compound condition testing:** In room_upgrade(), conditions like room == 'standard' and is_member were tested under (True, True), (True, False), and (False, True/False) scenarios.
- **Boundary condition testing:** Nights = 6 vs 7 (long-stay threshold), days_before = 1 vs 2 vs 7 (refund tiers), guests = 2 vs 3 (surcharge threshold), and rooms_free = 0 vs > 0.



5. Summary Coverage Metrics Table

Metric / Phase	Statements	Missed Stmts	Branches	Partial Branches	Coverage %
Initial Starter Tests	32	18	22	6	**37% (Failing)**
Code Coverage Suite	32	0	22	0	**100% (Passed)**
Branch Coverage Suite	32	0	22	0	**100% (Passed)**